OCP GPU & ACCELERATOR MANAGEMENT

INTERFACES

Version 0.9

Authors: (in alphabetical order)

David Blocker, NVIDIA

James Bodner, NVIDIA

Shrikant Giridhar, NVIDIA

Deepak Kodihalli, NVIDIA

Choudary Maddukuri, Microsoft

Hari Ramchandran, Microsoft

Johan van de Groenendaal, NVIDIA

Linda Wu, NVIDIA

Justin York, Google

Executive Summary

Management of GPUs is not standardized, resulting in significant effort and time to onboard each new HW design. The lack of standardization is also a burden on suppliers who must accommodate varying requirements from their customers. This document describes industry standard formats and protocols that make it easier for CSPs (Cloud Service Providers) to onboard new GPU and accelerator designs with less toil and faster time to market while reducing manageability permutations for suppliers.

Table of Contents

Introduction

Advances in accelerator-based workloads are driving the need for ever-faster bring-up and deployment of new accelerator-based system designs. Different accelerator components from different vendors implement a wide variety of management interfaces with little commonality. This lack of commonality causes extra toil and increases time-to-market for deploying new, innovative designs.

This document describes a common and generic framework for managing GPU and accelerators by way of management interfaces designed using standards-based management protocols.

This document covers both discrete accelerator devices (e.g., PCIe® CEM cards, as per PCI Express Card Electromechanical Specification Revision 5.1, Version 1.0) and Universal Baseboard (UBB) designs (e.g., OCP OAI HGX).

Overview

The section specifies the interfaces for managing both Discrete and UBB design GPU/Accelerators.

Discrete Accelerator Devices Manageability

Discrete accelerator devices are GPUs packaged as standard CEM form-factor PCIe adapter. Discrete accelerator device management can be broken down into several categories, each of which tie-in to various industry standards.

Table 1 – DMTF Standard Protocols

Manageability Objective	Technology	Standard
Static discovery	IPMI FRU	Platform Management FRU Information Storage Definition (rev. 1.3)
Transport protocol	MCTP	DMTF DSP0236 revision 1.3.1 or later
Attestation	SPDM	DMTF DSP0274 revision 1.2.1 or later
Management capability discovery	PLDM Type 0	DMTF DSP0240 revision 1.1.0 or later
Base monitoring and control	PLDM Type 2	DMTF DSP0248 revision 1.2.2 or later
Accelerator monitoring and control	PLDM Type 2	DMTF DSP2061 revision 1.0
Firmware management	PLDM Type 5	DMTF DSP0267 revision 1.2.0 or later
File I/O	PLDM Type (Future/TBD)	*WIP* DMTF DSP0242

PLDM Set State	PLDM	DSP0249 revision 1.1.0

Static discovery

The initial, static discovery of devices seeks to obtain primarily immutable information about the discrete accelerator device. This is done by reading the contents of a dedicated FRU EEPROM contained within the device.

Transport Protocol

MCTP is the media independent protocol for communication among management controllers within a system. This section below describes the Bus owner relationship, the endpoint discovery, EID assignment, the bridges and the EID pool assignments and corresponding flows.

Attestation

Once static and MCTP endpoint discovery is complete, an important step is attestation of the device. SPDM (DSP0274) operates over MCTP as a standardized mechanism to authenticate hardware identity and measure firmware identity. Using SPDM, the management controller serves as an SPDM requestor and discovers and negotiates the security capabilities to be used by the responder (discrete accelerator device). The management controller then authenticates the responder. Lastly, the management controller requests firmware measurements from the responder.

Note: SPDM attestation may happen repeatedly at the discretion of the management controller and is not limited to when the discrete accelerator device is initially discovered.

Management capability discovery

Once the hardware discrete accelerator device has been authenticated and measured, continuous management begins by discovering the management capabilities of the device. For standards-based management, subsequent discovery of the device capabilities happens by way of PLDM Type 0 (base) discovery.

Base monitoring and control

Once basic manageability capabilities are discovered dynamically from the device, continuous monitoring and control is performed by way of PLDM Type 2 (DSP0248).

Firmware management

Standards-based device firmware management requires support for PLDM Type 5 (DSP0267). DSP0267 provides a standardized way to query the version of the device's firmware (inventory) and to push firmware to the device (update).

UBB Accelerator Devices Manageability

Universal baseboard (UBB) designs are multi-GPU/accelerator devices with high-speed intra-GPU interconnects, PCIe switches and retimers, and complex topologies. In those HW devices the expectation is the UBB must present a Redfish ifnterface. The Redfish interface may be exposed by accelerator management controller (AMC) on or adjacent to the UBB or, in the future, via a software library (virtual AMC) provided by the supplier. (Details to be defined in v1.0.)

UBB designs may additionally support MCTP for high-frequency telemetry (e.g. thermal loop data) and attestation.

Table 2 – DMTF Standard Protocols

Manageability Objective	Technology	Description
Static discovery	IPMI FRU	Platform Management FRU Information Storage Definition (rev. 1.3)
Transport protocol	Redfish/MCTP	Redfish via NW and MCTP via I2C/I3C
Attestation	SPDM	Redfish and I2C/MCTP
Accelerator monitoring and control	Redfish/PLDM Type 2	Comprehensive monitoring via Redfish and High frequency telemetry via I2C/I3C
Firmware management	Redfish	Via Redfish

Static discovery

UBB accelerator designs are expected to present a FRU EEPROM (identical to what is required for discrete accelerator devices) and an AMC with a Redfish and limited MCTP and PLDM interface.

Attestation

SPDM attestation is managed by the BMC but control is effected through an AMC Redfish interface.

Base monitoring and control

UBB accelerator designs are managed predominantly through Redfish implemented by the UBB BMC. High frequency and critical telemetry like power and thermal are managed via MCTP/PLDM through a sideband connection like I2C/I3C.

Firmware management

UBB accelerator designs shall support firmware update through Redfish implemented by the AMC.

Discrete Accelerator Device Interfaces

This section lists the minimum set of protocols and OEM extension/format that any GPU/accelerator vendor needs to support on their discrete accelerator devices for hyperscalers to seamlessly manage these devices without any additional vendor/device specific work.

Access Frequency

The following table provides a high-level mapping of Data/Telemetry categories to protocols including the required frequency of access that must be reliably supported by the accelerator device.

Table 3 – Discrete Accelerator Management Protocols

Telemetry Data/Attribute (unordered)	Protocol	Use Case	Frequency
Monitoring (Sensors)	PLDM Type 2	e.g. Thermal, power…	1s
Inventory	MCTP/PLDM		On demand
Discovery	MCTP		On demand
Support Dump	Future direction: PLDM Type (DSP0242 - WIP)	Comprehensive triage	On demand
Debug Logs	Future direction: PLDM Type (DSP0242 - WIP)		On demand
Updates	PLDM Type 5		
Counters	Future direction: PLDM and MCTP OCP OEM	Tuning	"seconds"
Configuration/Settings	PLDM Type 2 (Effectors)		On demand

FRU Data

Static discovery of the discrete accelerator device is accomplished by accessing an I2C/I3C FRU EEPROM or, if not present, through virtual FRU devices. In the case of a virtual FRU, it must support the following requirements:

- Must be accessible on AUX power
- Needs a recovery mechanism if firmware implementing the virtual FRU fails

In either case the FRU device must return IPMI 1.3 FRU formatted data supporting the following requirements:

- Min 512 bytes, ideally 4KB (512byte min forces 2byte offsets)
- Internal Use Area: Optional
- Chassis Info Area: Optional
- Board Info Area: Mandatory (see below)
- Product info area: Mandatory (80 Bytes Minimum
- Multi Record Area: Mandatory for any OEM extensions and the last area so that it extends.

It is mandatory that a FRU EEPROM is only accessed by a single agent (e.g. BMC on Host Platform). Allowing multiple agents to access a FRU EEPROM can lead to FRU EEPROM data corruption and reduce system reliability.

The following table lists FRU areas that must be populated.

Table 4 – IPMI FRU Fields

FRU Area	Fields	Required w/ Valid Value
Common Header	Common Header	Y
Chassis Info Area	Chassis Type	N
	Chassis Part Number	N
	Chassis Serial Number	N
Board Info Area	Mfg Date/Time	N
	Board Manufacturer	Y
	Board Product Name	Y
	Board Serial Number	Y
	Board Part Number	Y
Product Info Area	Manufacturer Name	Y
	Product Name	Y
	Product Part/Model Number	Y
	Product Version	N

	Product Serial Number	Y
Multi-Record Area	Mandatory for any OEM extensions and the last area so that it extends.	Y, if applicable

MCTP

This section describes the MCTP discovery and inventory flows and lists minimum required commands. This section also proposes and defines a high-level generic VDM command format for data that needs OEM extensions.

Bus Owner and EIDs

Typically, the hyperscaler's BMC is the topmost bus owner and all other devices are endpoint devices. The endpoint devices can themselves be a bus owner when they are connected to 2 or more MCTP buses (also referred to as a "bridged device". The "topmost bus owner" is responsible for allocating ALL EIDs. It assigns a pool of EIDs to the bridge devices so that they allocate those EIDs to the device behind them.

MCTP Discovery

MCTP discovery/re-discovery happens on the following conditions:

- AC power ON
- DC power Cycle
- BMC reset
- Device Reset

Following an AC/DC power ON, the discovery can occur any time during the system boot and typically complete before the host operating system has booted. The end device should be ready in less than 60sec from power ON and the PCIe enumeration must not impact the I2C/I3C communication.

The flow charts below detail the MCTP discovery and EID assignment flows.

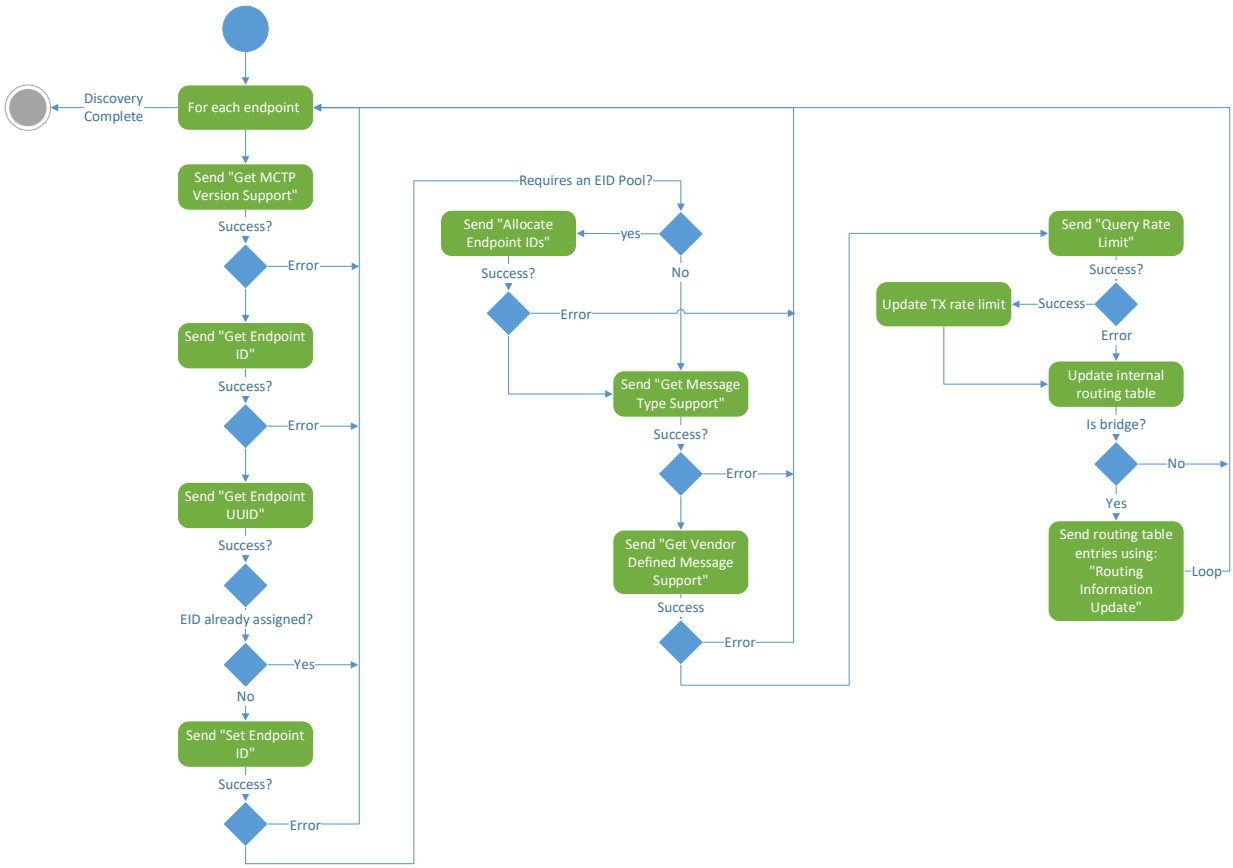

Figure 1 – MCTP Discovery Flow

MCTP command details

Table 4 lists the MCTP control commands and if they are required for use. This table is very similar to Table 12 within the DSP0236 specification with deviations indicated by superscripts.

Table 5 – MCTP Commands

Cmd Code	Command Name	Endpoint		Bridge		Notes
0x01	Set Endpoint ID	**Ma**	*Ng*	**Ca**[Ma]	**Mg**	
0x02	Get Endpoint ID	**Ma**	Og	**Ma**	**Og**[Mg]	
0x03	Get Endpoint UUID	**Ca**[1]	Og	**Ca**[1]	Og	Mandatory if device connects to multiple buses.
0x04	Get MCTP Version Support	**Ma**	Og	**Ma**	**Og**[Mg]	Bridges must use this command to verify compatible MCTP control protocol versions.
0x05	Get Message Type Support	**Ma**	Og	**Ma**	Og	
0x06	Get Vendor Defined Message Support	**Oa**[Ca1]	Og	**Oa**[Ca1]	Og	Mandatory to accept if device used vendor defined messaging.
0x07	Resolve Endpoint ID	*Na*	Og	**Ma**	Og	
0x08	Allocate Endpoint IDs	*Na*	Ng	**Ma**	*Mg*[Ng]	
0x09	Routing Information Update	Oa	Og	**Ma**	***Mg***[Ng]	
0x0A	Get Routing Table Entries	*Na*	Og	**Ma**	Og	
0x0B	Prepare for Endpoint Discovery	**Ca**[1]	*Ng*	**Ca**[1]	Cg[Ng]	Mandatory on a per bus basis to support endpoint discovery.
0x0C	Endpoint Discovery	**Ca**[1]	*Ng*	**Ca**[1]	Cg[Ng]	Mandatory on a per bus basis to support endpoint discovery.
0x0D	Discovery Notify	*Na*	**Cg**[1]	**Ca**[1]	**Cg**[1]	Mandatory if physical binding supports hot joins (e.g. I3C and PCI VDM).
0x0E	Get Network ID	Ca	Ca	Ca	Ca	
0x0F	Query Hop	*Na*	Og	**Ma**	Og	
0x10	Resolve UUID	*Na*	Og	**Ca**[Oa1]	Og	Mandatory if device and/or any connected devices connects to multiple buses.
0x11	Query rate limit	Ca[1]	Og	Ca[1]	Og	Mandatory if device can support more than 1 request at a time.
0x12	Request TX rate limit	Oa	Og	**Oa**[Ma]	Og	Bridges must allow for endpoints to change their rate limits
0x13	Update rate limit	Oa	Og	Oa	Og	

0x14	Query Supported Interfaces	Oa	Og	Oa	Og	

Key for Endpoint/Bridge columns:
Ma = mandatory to accept
Mg = mandatory to generate
Oa = optional to accept
Og = optional to generate
Ca = conditional to accept (see notes)
Cg = conditional to generate (see notes)
Na = not applicable to accept
Ng = not applicable to generate

SPDM

The OCP profile for accelerator attestation relies on the upcoming (post 1.0) version of the OCP Attestation of System Components v1.0 to define SPDM requirements for GPUs.

PLDM

This section describes how to model a GPU/accelerator PCIe adapter using PLDM Monitoring and Control protocol. It provides general guidelines on how to model the adapters and their components so that the CSPs can implement a common and generic framework for monitoring, control, and component firmware updates across devices and vendors.

The guidelines are based on the PLDM standards as defined by the DSP0248_1.1.0.

Discrete GPU/Accelerator Modules

The block diagram below is a generic representation of an accelerator adapter and can vary based on the vendor. Here, the discrete accelerator device represents the overall adapter and enclosed components as follows:

- SMC: This is the management controller (Satellite Management Controller) which implements PLDM and communicates to the Host BMC.
- GPUx: One or more GPU/accelerator controllers
- InterLink: one or more controllers that enable a high-speed connection between GPUs
- PCIe switch/controller

The block diagram also lists a subset of sensors/effectors that are supported by various components. The following sections describe how to model various sensors and controls via PLDM sensors, effectors and corresponding PDRs.

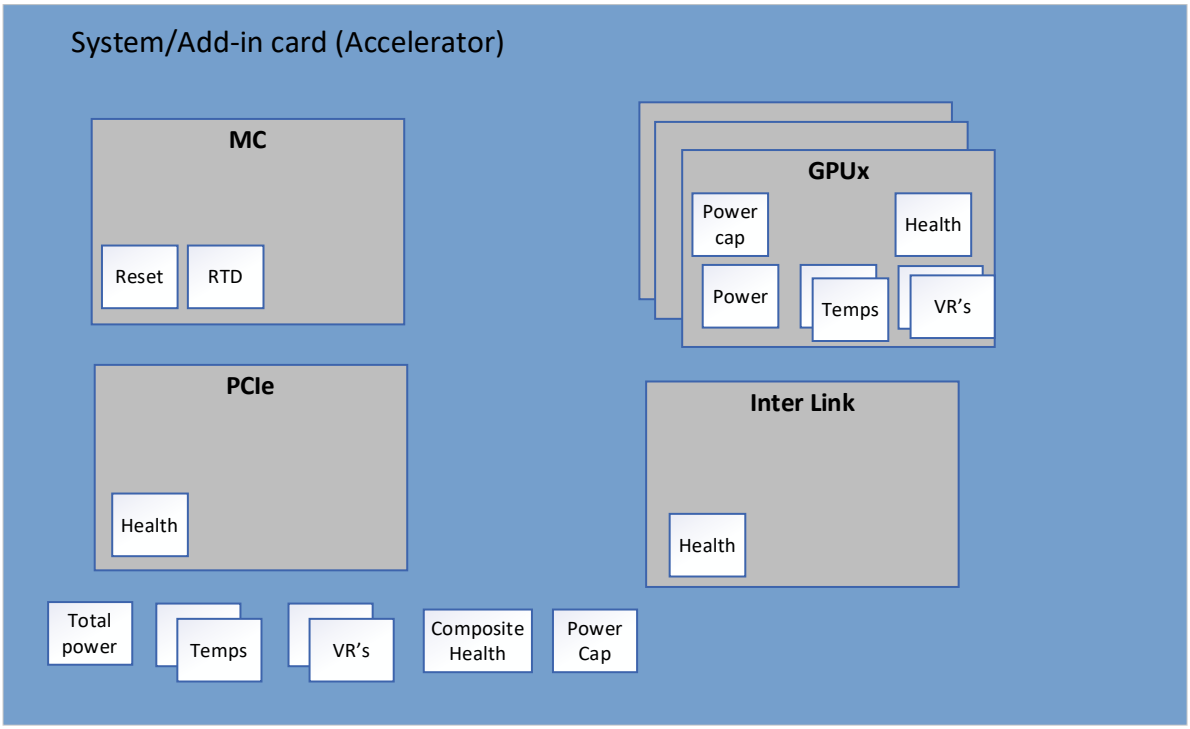

Figure 4 – PCIe CEM card

PLDM Sensors & Effectors

The section below provides high-level details of sensors and effectors and presents a minimum set of requirements.

Sensors

The implementation must support both numeric and state sensors. At a minimum the sensors listed below are required if present in the hardware design.

The temperature and power sensors are standard numeric sensors supported by the modules whereas the composite health sensor represents the overall health of the adapter.

Table 9 – PLDM Sensors and Types

Sensors	Sensor Type	Add-in Card	MC	GPU	Link	PCIe
Temp	Numeric	•		•	•	
Power	Numeric	•		•		
Total power	Numeric	•				
VRs	Numeric	•		•		
Composite Healthget	State	•				

Health	State	•	•	•	•	•

Effectors

The implementation must support both numeric and state effectors. At a minimum, the effectors listed below are required if supported by the hardware design.

The power cap effector must be supported at the discrete accelerator device (adapter) level and is optional at the GPU/accelerator module. The reset and reset to defaults (RTD) effectors must be supported by the management controller.

Table 10– PLDM Effectors

Effectors	Add-in Card	MC	GPU	Link	PCIe
Power Cap	•		•		
RTD		•			
Reset		•			

Entity Association

The following section describes how to model the entity association PDRs. These PDRs represent modules in a hierarchical model. The sections describe both physical and logical associations.

Physical Entity Association

These physical association PDRs are used to describe physical components and their association. The entity association PDR uses the following references to describe the modules and their association as defined in DSP0248:

- Container ID (an opaque number)
- Contained ID (an opaque number)
- Entity type: The entity type ID are as defined in DSP0248
- Entity instance: Module instance such as “GPU 0” or “GPU 1”.

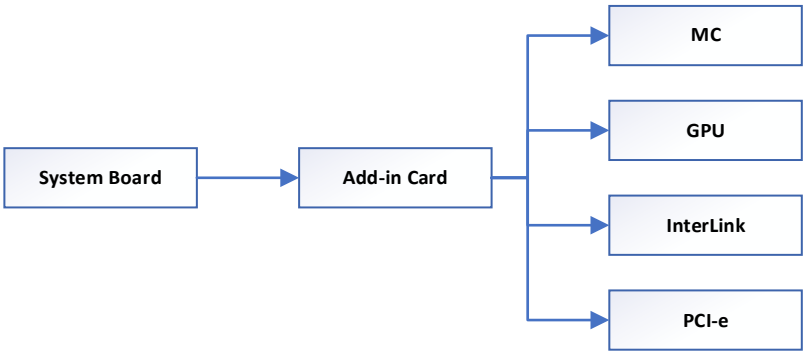

Figure 5 – Component Hierarchy

In the Figure below, the PDRs on the left side and the right side represent the same component hierarchy except via a single PDR vs multiple PDRs; either implementation is valid.

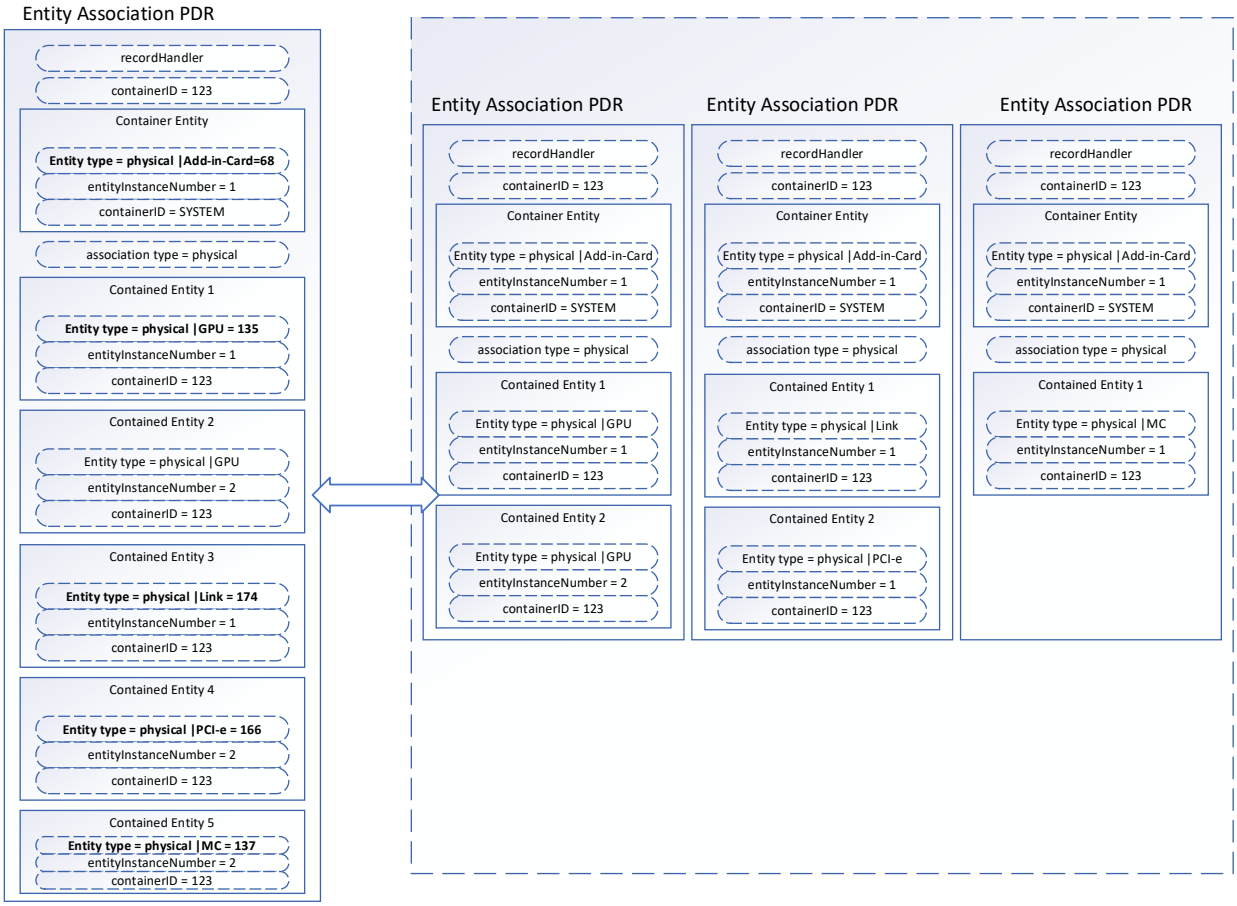

Figure 6 Entity Association PDRs

Logical Entity Association

Logical PDRs are used to associate a set of physical components to a logical entity. In this example the logical entity is “cooling domain”. This helps to group a set of sensors (in this example temperature sensors) of various physical modules to a single logical entity. This will help the management controller to associate these groups of sensors to the thermal algorithms used for fan and/or power control without requiring prior knowledge.

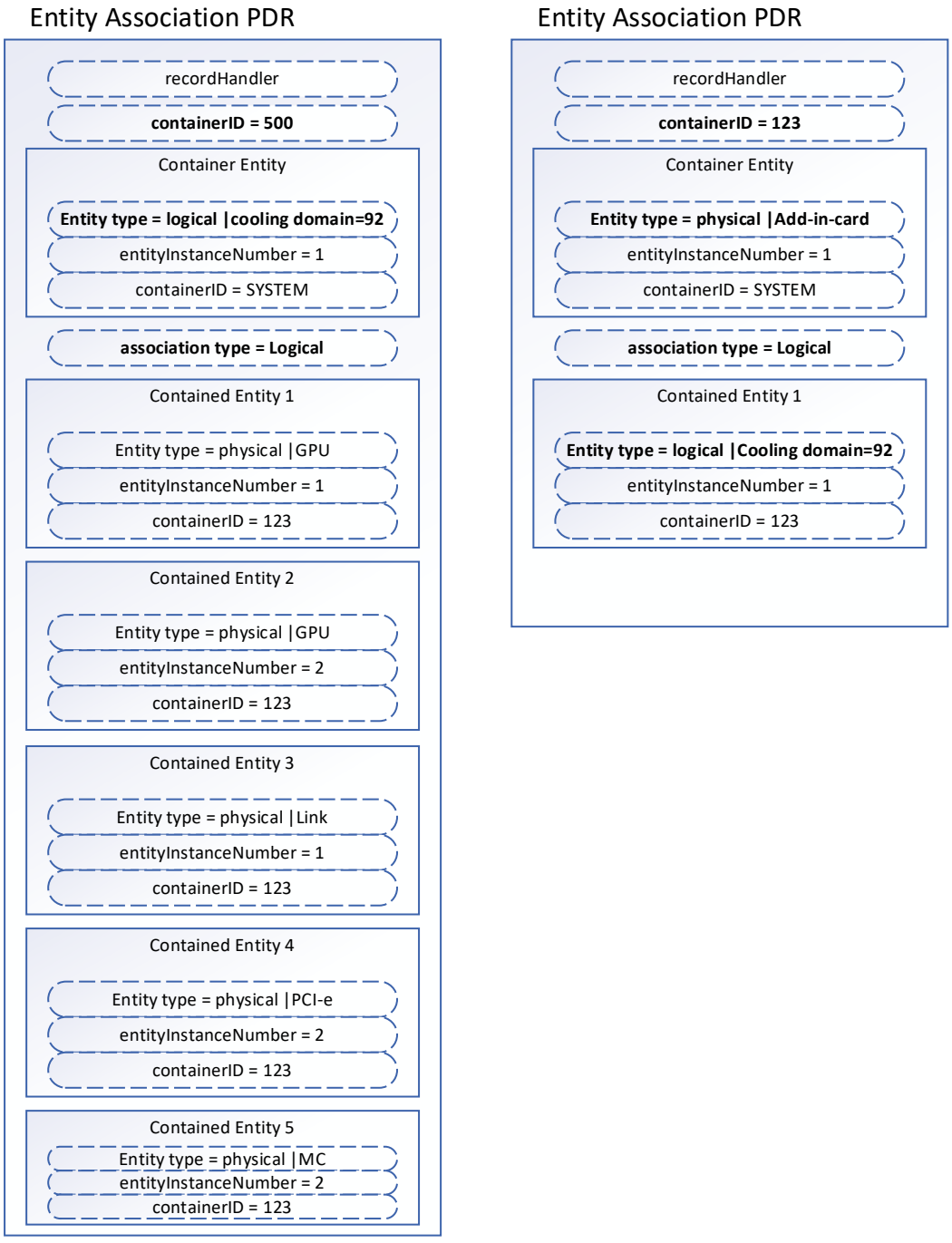

Figure 7 Logical Entity Association

Sensor & Effector Entity Mapping

The following section provides a list of examples showing how to map various sensors and effectors to entity association PDRs so that the Host BMC can map the sensors to the actual physical modules.

Numeric sensors

In the following example, a temperature sensor with sensor ID 10 belongs to GPU 1 while sensor ID 20 belongs to an InterLink module.

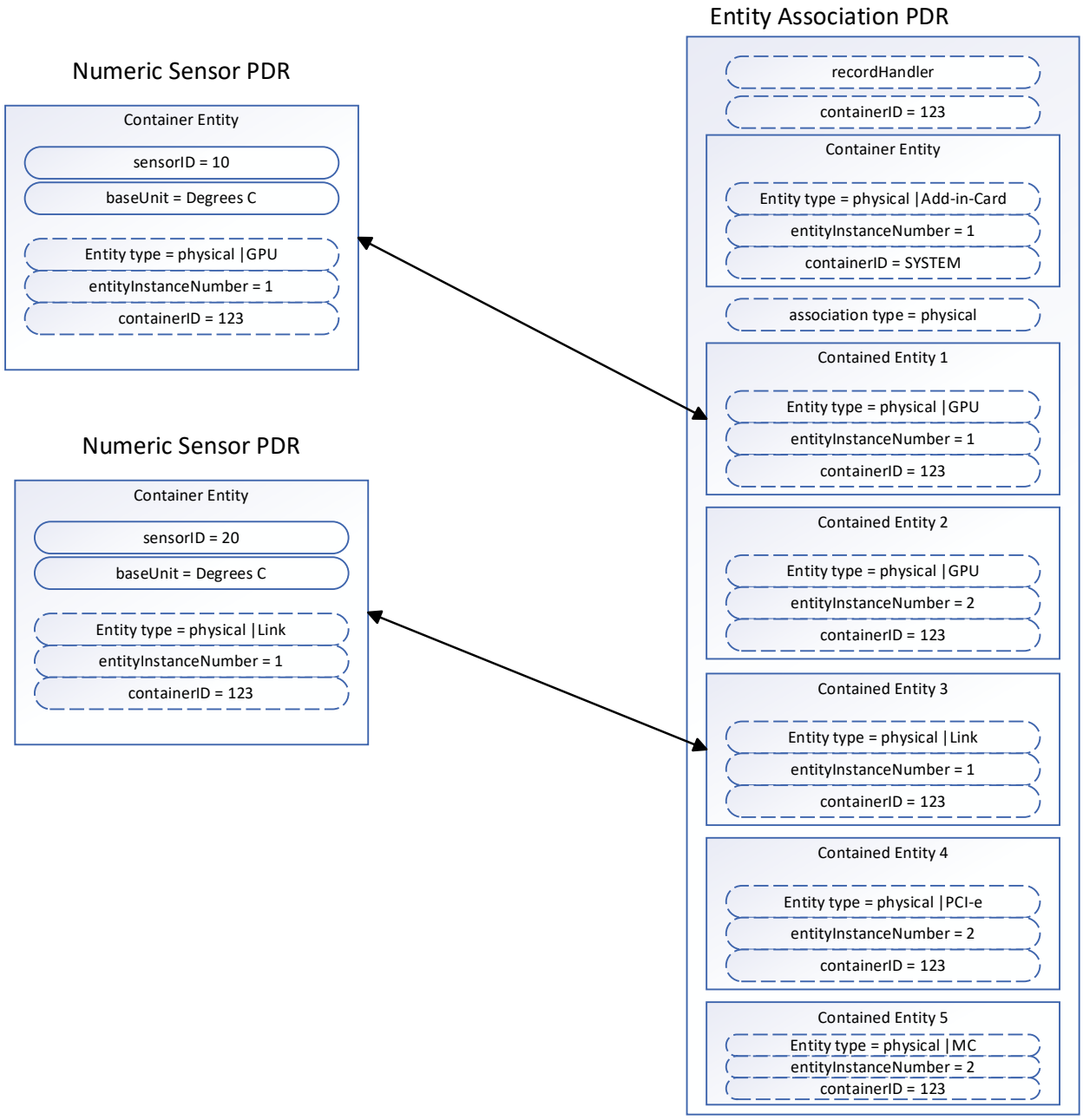

Figure 8 Numeric Sensor PDRs

State sensors

In this example, the Health state sensor is associated to an Add-in card to represent the overall health.

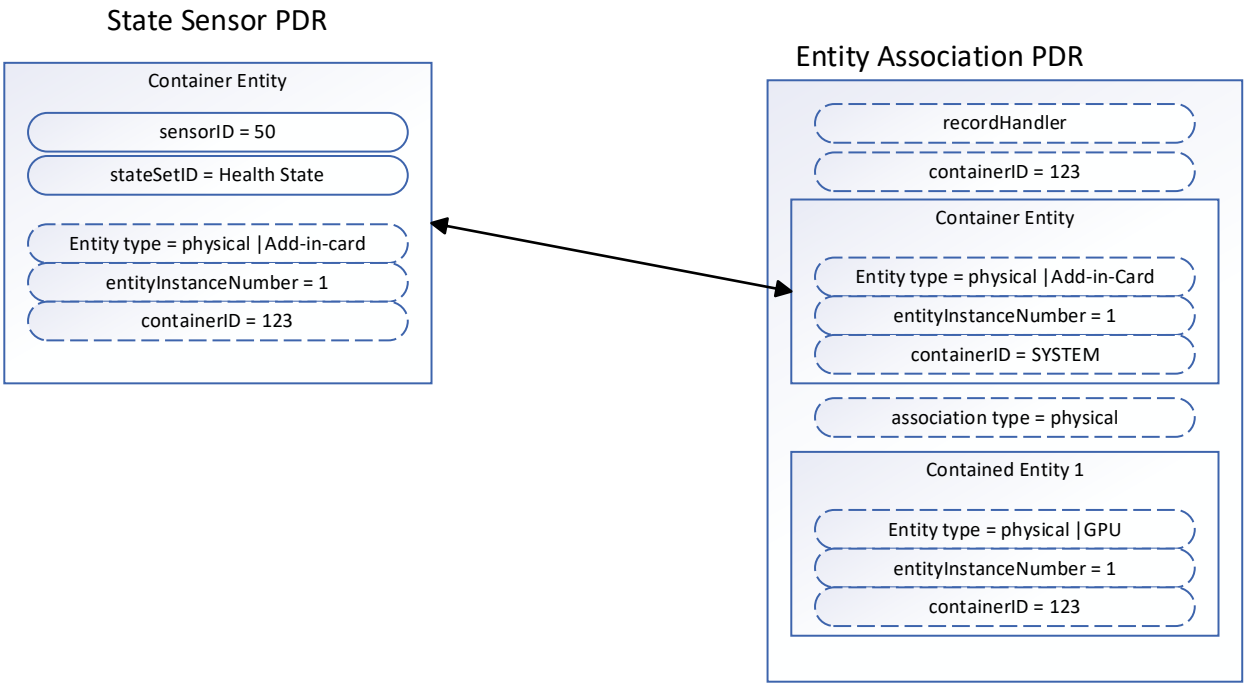

Figure 9 State Sensor PRD

Effectors

In this example, the numeric effector with ID 50 represents the overall power cap at the Add-in card level.

Likewise, the State Effectors “Reset” and “Reset to defaults” are associated to the management controller.

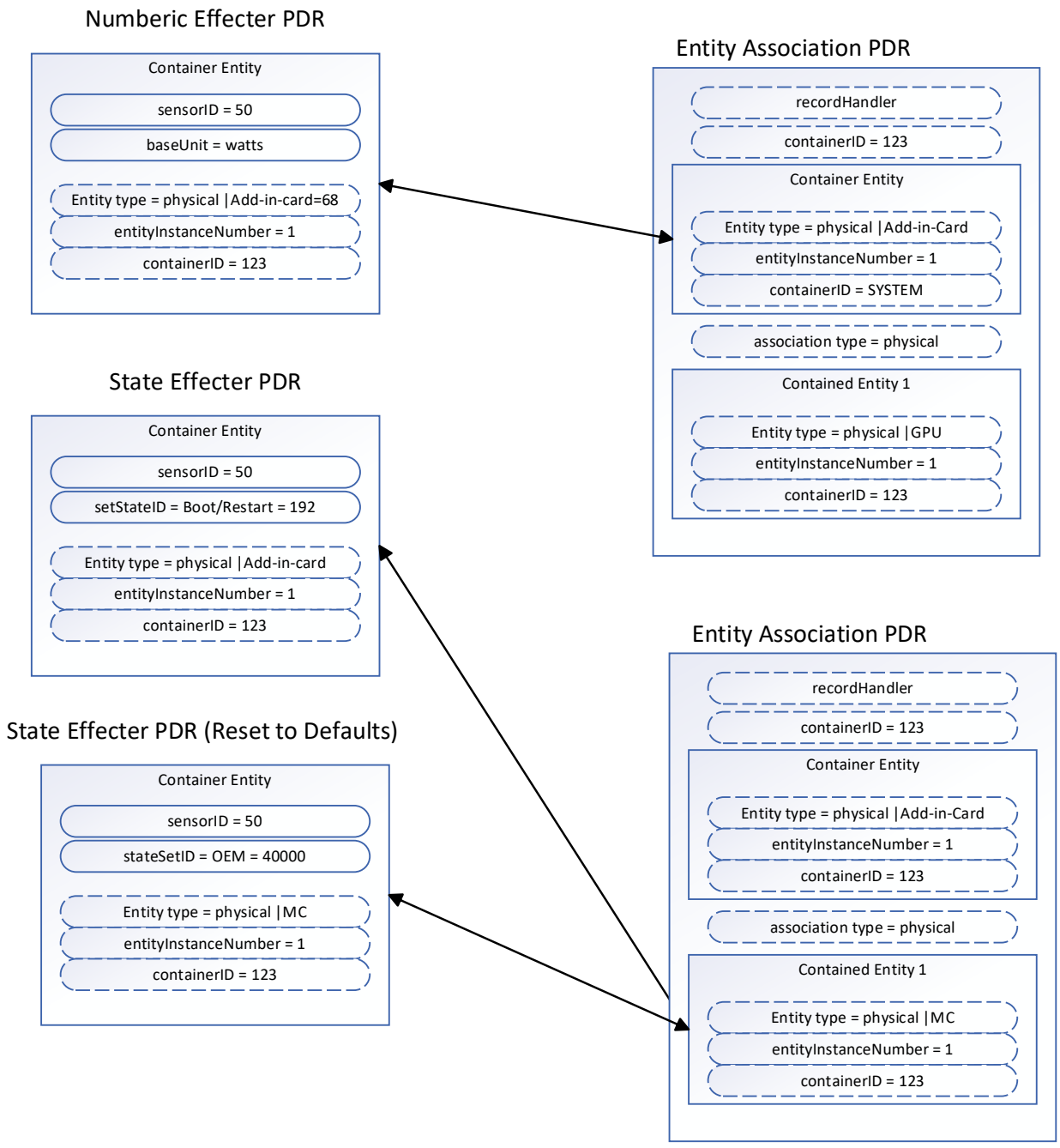

Figure 10 Effecter PDR

Logical sensor mapping

The mapping below associate sensor ID 15 of GPU 1 and sensor ID 25 of the InterLink module to the cooling domain entity.

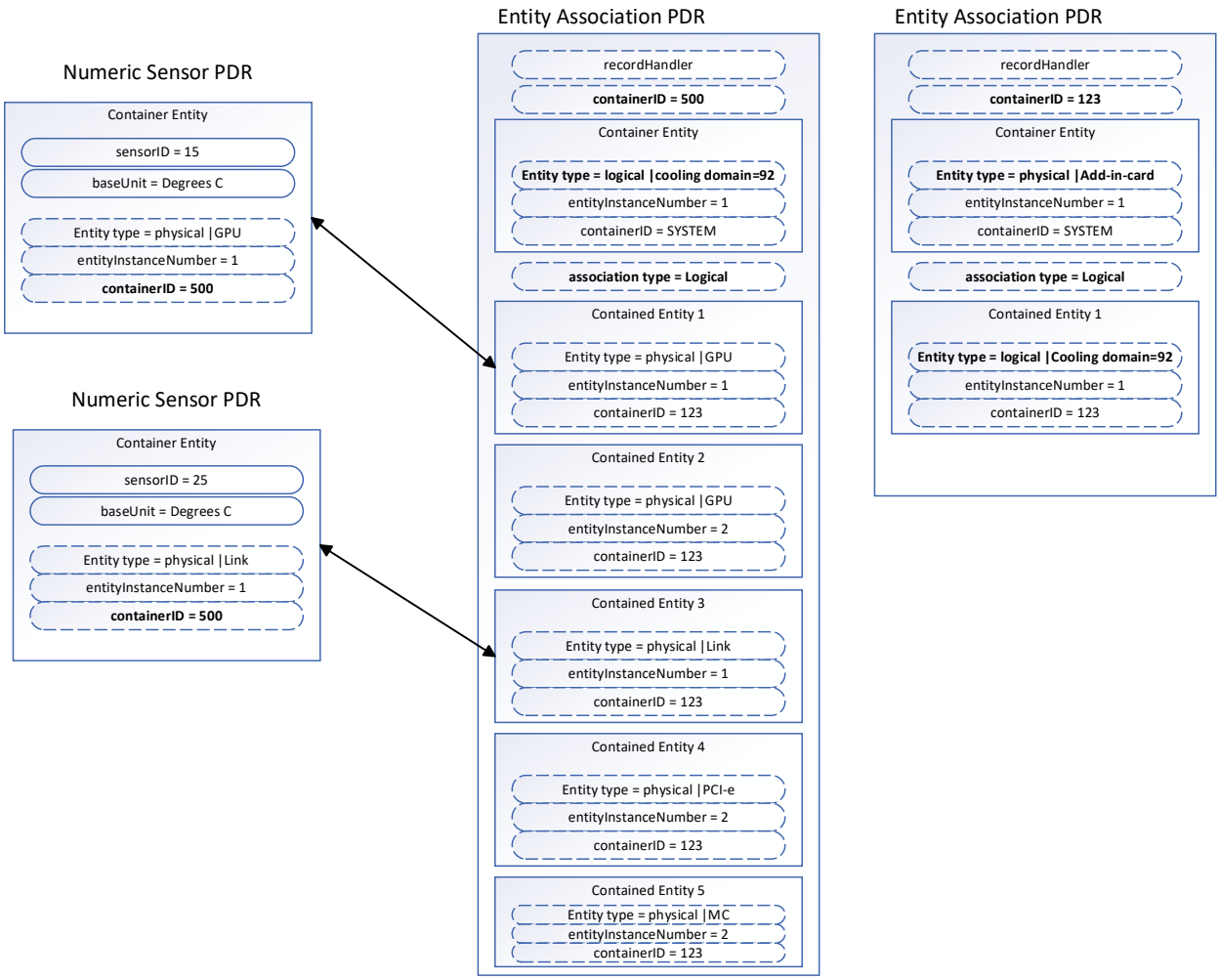

Figure 11 Logical Sensor Mapping PDRs

PLDM Type IDs

The following table lists the Type IDs used to represent the Accelerator adapter as per DSP0249.

Table 11 – PLDM IDs

Component	Entity	Entity ID
Management Controller	Management controller	137
Add-in card	Add-in card	68
GPU	Processor	149
InterLink	Inter-processor bus	174
PCIe	PCI Express Bus	166

MCTP OEM VDM

This section lists a reference implementation for how a device may expand telemetry data over MCTP when existing PLDM protocols are insufficient. It should be expected that device vendors will have additional OEM telemetry that will be harvested by the management controller that doesn't meet the existing definition of the PLDM Type 2 ecosystem. Fundamentally, the OCP group desires to push PLDM DMTF standards as quickly as possible but acknowledges that an OEM path for passing data is unavoidable.

Management Protocol Structures

This section describes the OCP VDM binding to the MCTP specification based on MCTP PCI vendor defined messages. The subsequent subsections describe the request, response and event message structures relative to the OCP VDM binding shown below.

Conventions

- **Byte order**: All protocol structures below follow little endian byte order.

OCP VDM binding

Bytes	Byte 1								Byte 2								Byte 3								Byte 4							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
1-4	IC	Message type = 0x7E							PCI vendor ID																RQ	D	Rsvd	Instance ID				
5-8	OCP	OCP Type			OCP Version				Vendor Message Type								Message payload															

Field Descriptions

Field Code	Field Size	Description
IC	1 bit	IC. Integrity check.
Message type	7 bits	Message Type = 0x7E.
PCI vendor ID	2 bytes	PCI defined vendor ID
RQ	1 bit	Request bit. This bit is used to differentiate between requests and other kinds of messages. This bit will be set to **1** for requests and events and to **0** for other messages.

D	1 bit	Datagram bit. This bit is used to indicate whether the instance ID is being used for asynchronous notifications (events) or for request/response tracking. This bit will be set to **1** for asynchronous notifications (events) and **0** for requests and response messages. The following table describes valid RQ and D bit combinations. RQ = 0, D = 0 – Response message RQ = 0, D = 1 – Event acknowledgment RQ = 1, D = 0 – Request message RQ = 1, D = 1 – Event message
Rsvd	1 bit	Reserved field. Must be zero.
Instance ID	5 bits	Instance ID. This field is used to uniquely identify a new instance of a request sent to the same MCTP endpoint and to match a particular instance of a request with its response. Retries shall carry the same instance ID as the original request.
OCP	1 bit	OCP designator. For all conformant OCP VDM messages this bit would be set to 1.
OCP Type	4 bits	OCP management interface type. This field shall be **0** for OCP VDM messages.
OCP Version	3 bits	OCP management interface version. This field shall be **1** for OCP VDM messages.
Vendor Message Type	1 byte	Represents the namespace for a group of related messages.
Message payload	Variable	Message structures. See the subsections below for details.

The following subsections describe the request, response and event message structures. The message structure diagrams below are relative to the OCP OEM binding described above which occupies the first 6 bytes of the message.

Request Message Format

Bytes	Byte 1								Byte 2								Byte 3								Byte 4							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
1-4	OCP message binding																Vendor Command Code								Data Size							
5-N	Data																															

Field Descriptions

Field Code	Field Size	Description
Vendor Command Code	1 byte	Vendor command code. Specifies the operation to be performed. Command codes are split into namespaces based on the Vendor Message Type field defined in the OCP MCTP binding.
Data size	1 byte	Size of the trailing command-specific data in bytes. For requests which do not require an additional data payload this field must be 0.
Payload	variable	Optional command-specific data.

Response Message Format

The response message structure follows one of the following formats:

- If a non-aggregate request was completed successfully, the response message will include a completion code of SUCCESS followed by data size and optional data bytes.
- If an aggregate request for multiple telemetry samples was completed successfully, the response message will include a completion code of SUCCESS followed by a telemetry count and a series of telemetry fields.
- If the request was unsuccessful, the completion code will be followed by a Reason code field which may contain extended information about the failure.

The message formats are shown below with their respective variants in the subsections below.

Response message with data

Bytes	Byte 1								Byte 2								Byte 3								Byte 4							
	7	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
1-4	OCP message binding																Vendor Command Code								Completion Code = SUCCESS							
5-N	Data Size																Data															

Fields Descriptions

Field Code	Field Size	Description
Vendor Command Code	1 byte	OEM Command Code for which this message is the response.
Completion code	1 byte	This field contains the status of the requested operation. See the list of defined Completion Codes table below.
Data size	2 bytes	Data size of the response payload in bytes. For Aggregate command response this is Telem Count
Data	variable	Command-specific response data.

Response message with reason code

Bytes	Byte 1							Byte 2								Byte 3								Byte 4							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
1-4	OCP message binding																Vendor Command Code								Completion Code							
5-8	Reason code																															

Fields Descriptions

Table 16 – Response Message Field Descriptions

Field Code	Field Size	Description
Vendor Command Code	1 byte	Vendor Command Code.
Completion code	1 byte	Status of the requested operation. See the list of defined Completion Codes table below.
Reason code	2 bytes	This field contains the command-specific reason code. This field expands on the status returned in the Completion code field before it.

Response message for bulk telemetry

Devices may choose to implement the follow protocol structure for aggregating multiple telemetry sources into a single response. In this scheme, each data source is assigned a unique tag and length which are used to identify the telemetry sample which they precede. A response may consist of multiple such telemetry records.

Bytes	Byte 1							Byte 2								Byte 3								Byte 4							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
1-4	OCP message binding																Vendor Command Code								Completion Code = SUCCESS							
5-8	Telemetry count																Tag								Rsvd				Length			V
9-N	Data (2^{Length} bytes)																															
N+	Other telemetry records follow																															

Fields Descriptions

Table 18 – Response Message Fields Description

Field Code	Field Size	Description
Vendor Command Code	1 byte	OEM Command Code.
Completion code	1 byte	Used to return the status of the requested operation.
Telemetry count	2 bytes	Count of the number of samples returned by the aggregate request.
Tag	1 byte	Unique tag that identifies the telemetry sample.
Rsvd	4 bits	Reserved for future use. Must be 0.
Length	3 bits	Length of the data sample in bytes as a power of 2.
V	1 bit	Valid bit. If the valid bit is unset the subsequent data shall be ignored by the requestor as invalid.
Data	Variable	Telemetry data.

An example of the aggregate response to combine temperature samples is shown below. The example message encodes the following telemetry samples into a single response:

Tag	Name	Data	Length (Len)	Valid (V)
0	Sensor 0 temperature	T0	4 bytes	Y
1	Sensor 1 temperature	T1	4 bytes	Y

3	Sensor 3 temperature	T3[1]	2 bytes	N
5	Sensor 5 temperature	T5	4 bytes	Y

Byte 1								**Byte 2**								**Byte 3**								**Byte 4**							
7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Telemetry count = 4																Tag = 0								Rsvd				Len = 2			V=1
T0 (byte 0)								T0 (byte 1)								T0 (byte 2)								T0 (byte 3)							
Tag = 1								Rsvd				Len = 2			V=1	T1 (byte 0)								T1 (byte 1)							
T1 (byte 2)								T1 (byte 3)								Tag = 3								Rsvd				Len = 1			V=0
T3 (byte 0)								T3* (byte 1)								Tag = 5								Rsvd				Len = 2			V=1
T5 (byte 0)								T5 (byte 1)								T5 (byte 2)								T5 (byte 3)							

Completion Codes

Table 16 – Completion Codes

Value	Name	Description
0x0	SUCCESS	The command completed successfully.
0x1	ERROR	Generic error code. This code shall not be returned when a specific error code is available.
0x2	ERR_INVALID_DATA	The request payload contained invalid data or illegal value.
0x3	ERR_INVALID_DATA_LENGTH	The requested data length does not match command specification (too long or too short).
0x4	ERR_NOT_READY	The responder is in a transient state where it is not ready to process the request.
0x5	ERR_UNSUPPORTED_COMMAND_CODE	The command code is not supported for this Vendor message type.
0x6	ERR_UNSUPPORTED_MSG_TYPE	The Vendor message type used in the request is not supported by the responder.
0x7 – 0x7E	RESERVED	Reserved for future use.
0x7F	ERR_BUS_ACCESS	Responder encountered an internal bus error.
0x80 – 0xFF	Command specific	This range of completion code values is reserved for values that are specific to a particular NSM request message.

[1] Invalid value.

Reason Codes

Value	Name	Description
0x0	NONE	No additional reason code is available

OEM Events

Events are asynchronous notifications that are issued by the managed device. The following message format will be used by the endpoint initiating the event.

Bytes	Byte 1								Byte 2								Byte 3								Byte 4							
	7	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
1-4	OCP message binding																Rsvd			AC KR	Event version				Event data size							
5-8	Event ID								Event data																							

Fields Descriptions

Field Code	Field Size	Description
Rsvd	3 bits	Reserved for future expansion.
ACKR	1 bit	Acknowledgement requested. This field may be used to request an optional ACK for event delivery: 0 – Acknowledgement message is not required 1 – Acknowledgement message is required
Version	4 bits	Event payload version.
Data size	1 byte	Size of the event payload (Event ID + Event Data) in bytes. Note that this field must be at least **1** to account for the fixed Event ID field.
Event ID	1 byte	Event identifier. This field is unique within a given Vendor Message Type. Multiple events with the same Event identifier may be distinguished using the Instance ID field in the OCP binding.
Data	Variable	Event-specific payload.

OEM Event Acknowledgement

By default, events are unacknowledged datagram messages and do not require any action on behalf of the management controller.

However, the device may support a method of generating acknowledged events for e.g., to require delivery guarantees for critical events. In such cases, the device may request an acknowledgement (ACK) from the management controller upon receipt of the event message. This is done using the ACKR flag in the Event message body.

The following requirements apply for acknowledged event messages:

- Restrict to only one event receiver.
- There can only be a single event waiting for an ACK at a time. Unacknowledged events can continue to be generated.
- When an ACK is not received with the acknowledgment timeout duration the device must retry delivery of the event message up to NRETRY times. If a message cannot be delivered after this retry period, the event is considered expired and may be dropped by the device.

The following message format is used for ACK messages which are differentiated based on the RQ and D fields in the OCP binding.

Bytes	Byte 1								Byte 2								Byte 3								Byte 4							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
1-4	IC	Message type = 0x7E							PCI vendor ID																RQ	D	Rsvd	Instance ID				
5-8	Event ID																															

Field Descriptions

Field Code	Field Size	Description
RQ	1 bit	Request bit. This bit is used to differentiate between requests and other kind of messages. Would be set to 1b for request messages and event messages. Would be set to 0b for response messages.
D	1 bit	Datagram bit. This bit is used to indicate whether the instance ID is being used for asynchronous notifications (events) or for request/response tracking. This bit shall be set to 1 for event ACK messages.

Rsvd	1 bit	Reserved. Must be 0.
Instance ID	5 bits	Used to uniquely identify a new instance of the API request. This is used to differentiate requests sent to the same MCTP endpoint and to match a particular instance of a request with a corresponding instance of the response.
Event ID	1 byte	Event identifier. This field is unique within a given Vendor Message Type. Multiple events with the same Event identifier may be distinguished using the Instance ID field in the OCP binding.

UBB Accelerator Device Interfaces

This section lists the minimum set of protocols and OEM extension/formats/schemas that any GPU/accelerator vendor needs to support on their AMC so that hyperscalers can seamlessly manage these devices without any additional vendor specific work.

Redfish

This section describes the high-level overview of Redfish organization. This section also proposes various Redfish resource behaviors and OEM schema extensions.

Redfish Tree

The Redfish tree diagram offers a comprehensive overview of the AMC management capabilities and resource organization. It provides an overview and relationships between different components. The resource label “UBB” represents the complete Accelerator subsystem/enclosure within the server and the resource label “OAM_X” represents each of the Accelerator module assembly.

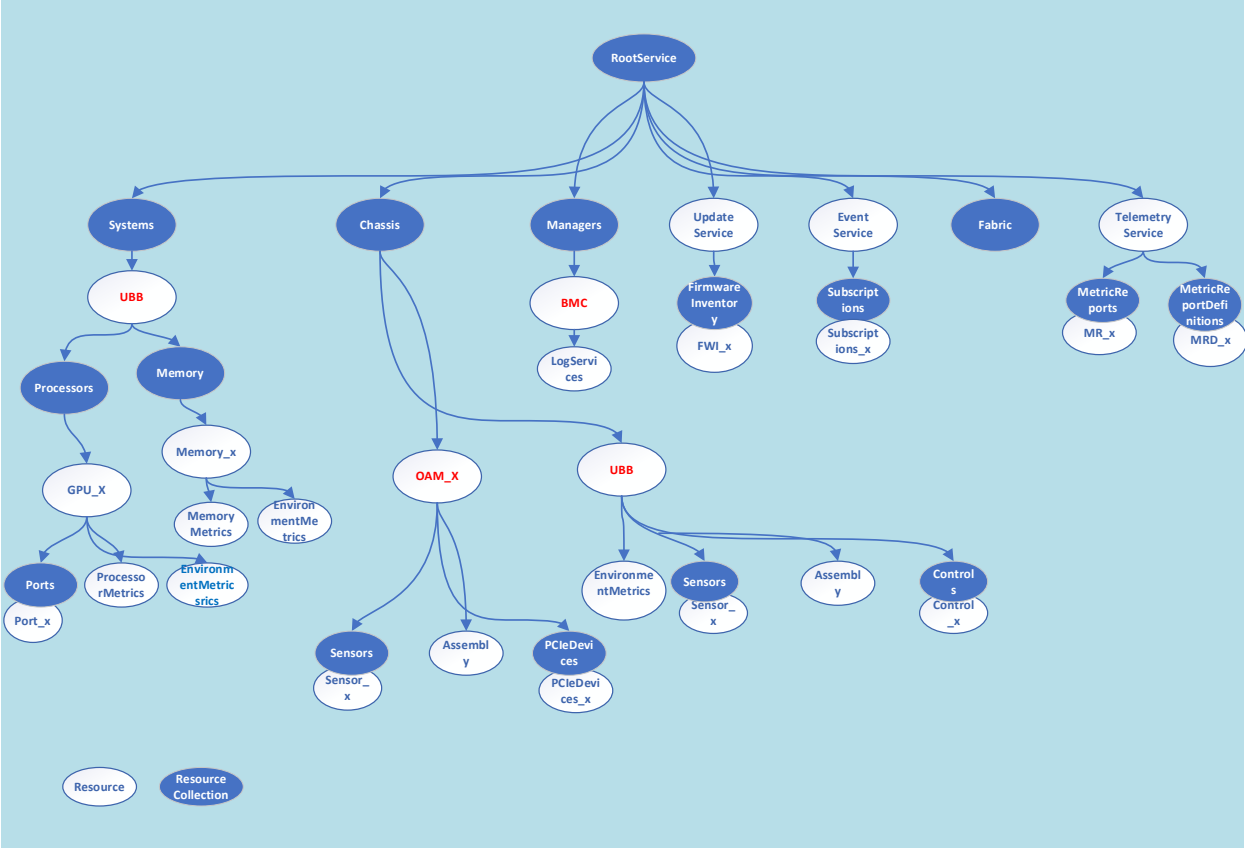

Figure 12 UBB Redfish Model

Location Objects

The Location object referenced from many Redfish resources is very important in enabling automation at scale. CSPs use various techniques to correlate the contents of the Location object into unambiguous mappings to physical hardware in their data centers.

Table 17 – Location Object Example

```
"Location":
{
  "PartLocationContext": "Backplane 3, Slot 4",
  "PartLocation":
  {
    "LocationType": "Port",
     "ServiceLabel": "Port 1"
  }
}
```

Location property example.

CSPs often rely on a tuple formed by the combination of the `Location.PartLocationContext`, and `Location.PartLocation` to bind entities to external logical management controls. For this reason, the Location resource, and its `PartLocationContext`, `PartLocation.LocationType`, and `PartLocation.ServiceLabel` fields are required by CSPs in any resource that supports `Location`.

RAS Error Injection

The RAS error injection is the capability to inject various HW errors at the IP level, SOC level and/or at the platform level. This information is described in greater detail in the OCP GPU & Accelerator RAS Requirements specification. The document details the Redfish APIs and the necessary properties and schemas.
Link:

Out-of-Band Interfaces (MCTP, PLDM, and SPDM)

UBB designs are connected to the enclosing system via high-speed networking, where Redfish is the interface, and out-of-band connections including I2C/I3C, where MCTP, PLDM, and SPDM are the interface protocols.

The implementation of the out-of-band protocols is same as with discrete GPU/Accelerator devices described in the above sections except that it out-of-band communication is limited to specific purposes such as monitoring high-frequency telemetry for thermal and power related features.

Redfish Interoperability Profile (RIP)

The GPU management Redfish Interoperability Profile (RIP) follows the DMTF Redfish Interoperability Profile standard and provides specifics on the minimum set of Redfish resources and properties required by hyperscalers when managing a UBB-based GPU design.

A supplier may run the OCP UBB GPU RIP to confirm their accelerator management controller's (AMC) Redfish implementation supports the minimum set of Redfish required to ease integration with hyperscale environments. It should be understood that this RIP is focused on hyperscaler requirements and does not limit the Redfish implementation that a supplier may provide for other classes of customers.

Conclusion

The management profiles for GPUs in this document provide guidance for how GPU devices can be designed and how CSPs can manage them. These profiles reduce the amount of work required for GPU suppliers to bring new products to market by converging CSP requirements around industry standards. Likewise, these profiles reduce the effort and time required for CSPs to bring new GPU designs to market by enabling a higher level of code and test re-use between different GPU parts and vendors.

Glossary

AMC– Accelerator Management Controller; the Universal Base Board Management Controller. This is the Redfish controlled management controller present on an OCP OAI HGX UBB board.

BMC – Baseboard Management Controller. This is a management controller present within system designs that frequently interacts with the management of discrete accelerator devices and UBBs.

CSP – Cloud Service Provider

Hyperscaler – Cloud Service Provider

UBB – Universal baseboard; a type of accelerator delivered as a large board containing multiple GPUs and high-speed fabric interconnect.

References

- OCP GPU & Accelerator RAS Requirements v0.5
- OCP Attestation of System Components
- Platform Management FRU Information Storage Definition rev. 1.3 (IPMI)
- DMTF DSP0236 revision 1.3.1 or later (MCTP)
- DMTF DSP0274 revision 1.2.1 or later (SPDM)
- DMTF DSP0240 revision 1.1.0 or later (PLDM Type 0)
- DMTF DSP0248 revision 1.2.2 or later (PLDM Type 2)
- DMTF DSP0267 revision 1.2.0 or later (PLDM Type 5)
- DMTF DSP0266 revision 1.18.0 or later (Redfish)

License - Open Web Foundation (OWF) CLA

Google, Microsoft, NVIDIA

Usage of this Specification is governed by the terms and conditions set forth in **Open Web Foundation Modified Final Specification Agreement ("OWFa 1.0.2") ("Specification License").**

You can review the applicable OWFa1.0 Specification License(s) referenced above by the contributors to this Specification on the OCP website at http://www.opencompute.org/participate/legal-documents/. For actual executed copies of either agreement, please contact OCP directly.

Notes:

1. The above license does not apply to the Appendix or Appendices. The information in the Appendix or Appendices is for reference only and non-normative in nature.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP "AS IS" AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY DAMAGES WITH RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION,

WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

OCP Tenets

Openness

- This specification was developed via close and open collaboration between industry partners and competitors.
- All specifications and interfaces produced through this effort will be available to all OCP members.

Efficiency

- The goal of this specification is to make integration of GPUs into Hypercaler solutions seamless, reducing toil for both the supplier and the hyperscaler consumers.
- A companion to this effort is an OCP compliance tool that will enable automated validation of these interfaces, ensuring reduced toil and high-quality products

Impact

- This document represents the first industry initiative to standardize GPU requirements between suppliers and hyperscale consumers.
- The advances in this document are expected to have significant impact on quality and time-to-market for GPU systems deployed by hyperscalers.
    - These advances will also be applicable and beneficial to enterprise deployments of GPU systems.

Scale

- This specification applies to very large-scale GPU system deployments in hyperscaler data centers.

Sustainability

- The profiles defined in this document enable cross-generational commonality for key functionality of GPU parts, enabling logistics to support longer lifespan of GPU parts and a healthy secondary market for these parts.

About Open Compute Foundation

At the core of the Open Compute Project (OCP) is its Community of hyperscale data center operators, joined by telecom and colocation providers and enterprise IT users, working with vendors to develop open innovations that, when embedded in product are deployed from the cloud to the edge. The OCP Foundation is responsible for fostering and serving the OCP Community to meet the market and shape the future, taking hyperscale led innovations to everyone. Meeting the market is accomplished through open designs and best practices, and with data center facility and IT equipment embedding OCP Community-developed innovations for efficiency, at-scale operations and sustainability. Shaping the future includes investing in strategic initiatives that prepare the IT ecosystem for major changes, such as AI & ML, optics, advanced cooling techniques, and composable silicon. Learn more at www.opencompute.org.